

1. PROCESSUS LÉGERS EN C

En C on dispose des fonctions suivantes dans `pthread.h` :

```
int pthread_create(pthread_t *thread,
    pthread_attr_t *attr,
    void *(*start_routine)(void *),
    void *arg);

int pthread_join(pthread_t thread, void **retval);
```

On utilisera bien souvent la fonction `pthread_create` de la façon suivante :

```
pthread_t thread;
pthread_create(&thread, NULL, une_fonction_a_lancer, NULL);
```

Question 1. Lire la documentation de ces deux fonctions. Pour la fonction `pthread_create`, comprendre le type attendu pour le troisième paramètre (et voir à quoi pourrait servir le quatrième).

On dispose également de *mutex* fournis par la bibliothèque `pthread`, avec le type `pthread_mutex_t`. Un mutex doit être initialisé, une seule fois, avant d'être utilisé, ce qui peut se faire de deux façons :

```
// Initialisation par affectation à la déclaration de la variable
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// Initialisation dans tous les cas
pthread_mutex_init(&mutex, NULL);
```

Un mutex se verrouille et se déverrouille dans chaque thread qui l'utilise avec :

```
pthread_mutex_lock(&mutex);
/* Section critique entre les deux */
pthread_mutex_unlock(&mutex);
```

Les programmes utilisant la bibliothèque `pthread` doivent être compilés avec l'option `-pthread` de `gcc` :

```
gcc -pthread -o source.o source.c
gcc -pthread -o executable source1.o source2.o source3.o
```

2. COMPTEUR DÉTRAQUÉ

Question 2. Écrivez un programme qui lance deux processus légers, qui incrémentent tous un même compteur partagé sans aucune précaution. Que constatez-vous ?

Dans un deuxième temps, ajoutez des `printf` à chaque incrémentation et recommencez le test : avez-vous une explication ?

Question 3. Programmer l'incrémentation du compteur avec un mutex.

3. BOITES NOIRES

On vous fournit des fichiers `boite_noire2.o`, `boite_noire2.h`, et un fichier d'exemple `test.c`.

On a des boîtes noires qui doivent exécuter un certain nombre de tâches le plus rapidement possible. Une boîte doit d'abord être démarrée avec un appel à `boite_demarre`, puis on exécute chaque tâche avec un appel à `boite_noire_tache`, puis on éteint la boîte avec `boite_eteint`.

Le nombre de tâches à traiter est donné par la fonction `boite_objectif` .
On doit faire cela le plus rapidement possible, l'ensemble de l'exécution est chronométrée.
Il est recommandé de lire attentivement le fichier `boite_noire2.h` .

3.1. BOITE 1. GESTION DE TÂCHES INDÉPENDANTES

Dans cette première partie, on utilise la boîte créée par `boite_cree_1()` ; . Cette boîte contient 100 tâches qui sont toutes indépendantes les unes des autres, on peut les exécuter dans n'importe quel ordre.

L'objectif de ce premier exercice est de faire un programme C qui fait un premier appel à la fonction `demarre_boite` puis fait, ici, 100 appels à la fonction `boite_noire_tache` puis un appel à la fonction `eteint_boite` .

L'objectif de l'exercice est de faire le code qui fait le plus rapidement possible les 100 appels à la fonction `boite_noire` sachant que chaque appel à `boite_noire_tache` prend un certain temps. Votre programme peut utiliser plusieurs threads pour accélérer le temps d'exécution mais il ne doit pas utiliser plus de 10 threads (en comptant le thread principal).

Pour cette exercice uniquement, on ne connaît pas la durée que met chaque tâche mais on peut supposer qu'une tâche met autant de temps à s'exécuter quel que soit le thread qui l'exécute et quel que soit ce que les autres threads font (peu importe qu'ils travaillent sur une tâche ou non). Les différentes tâches ne prennent pas forcément toutes le même temps à s'exécuter (certaines peuvent durer 5s et d'autres 0.5s).

Il est demandé de commencer par des solutions simples mais qui marchent puis d'améliorer progressivement vos solutions. On commencera donc par une version qui ne lance aucun thread supplémentaire pour s'assurer que l'on a bien compris le fonctionnement de la bibliothèque puis on pourra tester des versions qui lancent des threads avant d'écrire des solutions qui font des choses compliquées avec les threads lancés.

4. BOITE 2. GESTION DE TÂCHES AVEC PRÉREQUIS ET TEMPS DE CALCUL

Dans ce deuxième exercice, on utilise maintenant la boîte créée par `boite_cree_2()` ; . On a toujours un certain nombre de tâches à accomplir (ici, 1000) mais cette fois les tâches ne sont plus indépendantes et l'on a de l'information sur les tâches à accomplir.

Plus précisément, on a, pour chaque tâche, une liste de ses prérequis et le temps (en microsecondes) qu'elle prend. Quand une tâche A a une tâche B pour prérequis, il faut que B soit finie avant de pouvoir lancer A. De nouveau, on veut une solution qui finisse l'ensemble des tâches le plus rapidement possible et l'on se donne comme limite de ne pas utiliser plus de deux threads (en comptant le thread principal).

Utilisation de la bibliothèque. Consulter le fichier `boite_noire2.h` pour une documentation précise de la bibliothèque.

Recommandation. On essaiera d'abord d'avoir une solution correcte avant d'avoir une solution qui essaie d'utiliser des threads ou d'être optimale. Les fonctions de la boîte noire sont "thread safe" c'est-à-dire que vous n'avez pas besoin de prendre des mutex avant de les appeler.

5. BARRIÈRE DE SYNCHRONISATION

On veut lancer `n` threads et faire en sorte d'avoir une fonction `attendre_tous` qui garantit que tous les threads sont bloqués lorsqu'ils appellent `attendre_tous` , jusqu'à ce que le dernier thread appelle cette fonction. Dans ce cas, tous les threads sont débloqués.

Pour cela, on va utiliser des *sémaphores*. Un sémaphore est une structure de données de type `sem_t` qui contient un entier, qui ne peut jamais être négatif. La structure est déclarée dans `semaphore.h` . On a trois fonctions :

```
// Initialisation d'un sémaphore à la valeur donnée en troisième paramètre.
// On mettra toujours l'entier du milieu à 0.
int sem_init(sem_t *sem, int pshared, unsigned int value);

// Incrémentement de la valeur d'un sémaphore.
int sem_post(sem_t *sem);

// Décrémentement de la valeur. Si le sémaphore est à zéro, cette fonction
// bloque jusqu'à ce qu'un autre thread incrémente le sémaphore.
int sem_wait(sem_t *sem);
```

Question 4. Proposer une implémentation d'une barrière entre n threads. On pourra proposer une structure qui définit un type `barriere` ainsi qu'une fonction `attendre_tous`.

Question 5. Cette barrière est-elle réutilisable plusieurs fois de suite ? Si ce n'est pas le cas, proposer une barrière réutilisable.

6. DES BULLES

Le tri à bulles n'intéresse personne. Mais on peut imaginer une variante où on imagine qu'en chaque paire de cases consécutives du tableau à trier se trouve un lutin dont le travail est de permuter les deux cases lorsqu'elles ne sont pas ordonnées en croissant. Les lutins travaillent en parallèle.

Question 6. Les lutins doivent-ils se synchroniser ? Comment évaluer la terminaison ?

Question 7. Proposer un algorithme de tri à bulles par lutins parallèles.

Question 8. Programmer une fonction `tri_lutins` qui réalise ce tri. Est-ce rapide ?

7. DIVISER POUR RÉGNER, PARALLÈLEMENT

Le tri rapide se parallélise naturellement, étant donné que les appels récurifs sur chaque sous liste sont totalement indépendants.

Question 9. Écrire une fonction `tri_rapide_parallele_naif` qui effectue le tri rapide d'un tableau. Au lieu de faire un simple appel récursif, elle démarre un nouveau processus léger pour chaque traitement récursif de la sous liste.

Créer un processus léger demande un peu de temps de traitement, l'opération n'est pas si légère que cela. Il n'est pas vraiment rentable de séparer le traitement des sous listes lorsque leur taille est vraiment trop petite. De plus, le nombre de processus créés par la fonction précédente dépasse en général largement le nombre de processeurs présents sur la machine : ceci empêche d'obtenir un vrai gain en parallélisant.

Supposons que l'on dispose de p processeurs. Nous allons plutôt écrire une version qui se limite à créer p processus légers.

La phase de partition autour d'un pivot peut être parallélisée, en créant p processus légers. Le tableau de départ est découpé en p morceaux de tailles équivalentes. On choisit un pivot, qui est partagé par tous les processus légers. Chaque processus léger est chargé de découper le tableau en deux parties autour du pivot. On regroupe alors tous les éléments inférieurs au pivot d'une part, tous les éléments supérieurs d'autre part. Le résultat ne sera pas stocké sur place.

Les processus se divisent alors en deux groupes de $p/2$ processeurs et on recommence l'opération.

Question 10. Écrire une fonction `tri_rapide_parallele` qui effectue ce tri. Le nombre p pourra être donné en paramètre.